Panduan Esensial: Git, GitHub, & .gitignore

Rangkuman Praktis Cara Kerja Version Control, Manajemen Berkas, & Keamanan Kode Proyek

Catatan Awal: Berkas static/img/.gitkeep sengaja ditambahkan agar direktori gambar tim tetap ikut ter-commit ke repositori (Git secara default tidak mencatat direktori kosong). Tercatat pada commit 9d95518.

1. Perbedaan Mendasar: Git vs GitHub

Git: Alat version control engine lokal di komputer. Tugasnya mencatat histori setiap modifikasi file (siapa mengubah apa, kapan, dan kemampuan kembali ke versi lampau kapan saja). Dapat beroperasi 100% offline tanpa internet.

GitHub: Layanan cloud hosting untuk repositori Git di internet. GitHub bertindak sebagai *remote server* untuk cadangan data, kolaborasi tim, ulasan kode (Pull Request), manajemen tiket (Issues), dan pipeline deployment.

2. Istilah Inti yang Wajib Dipahami

- **Repository (Repo):** Folder proyek yang dipantau Git (memiliki direktori tersembunyi .git/).
- **Commit:** Snapshot permanen seluruh isi proyek pada titik waktu tertentu dengan pesan keterangan.
- **Branch:** Jalur pengerjaan independen terpisah. Jalur default standar bernama main.
- **Remote:** Alamat repositori online di GitHub, umumnya diberi nama alias origin.
- **Staging Area:** Ruang perantara untuk memilih berkas yang siap disimpan ke commit berikutnya (git add).
- **Untracked vs Tracked: Untracked:** berkas baru belum dipantau. **Tracked:** berkas yang sudah pernah dicatat perubahannya.

3. Memahami Berkas .gitignore

.gitignore adalah berkas teks polos di root repositori berisi daftar pola nama file/folder yang **wajib diabaikan** oleh Git secara permanen. File yang cocok tidak akan muncul di git status dan tidak akan terunggah ke repositori online.

Tiga Fungsi Utama: (1) Mencegah file sampah/log lokal masuk riwayat; (2) Mencegah file ukuran besar yang bisa di-generate ulang; (3) Mencegah kebocoran kredensial rahasia (API key, password).

```
nama.log         # Abaikan file 'nama.log' di folder mana pun
/config.json     # Hanya abaikan 'config.json' persis di root direktori
builds/          # Abaikan seluruh direktori 'builds/' beserta isinya
*.pyc            # Abaikan semua file berakhiran .pyc
!penting.log     # Pengecualian: file ini tetap dipantau walau ada aturan abaikan
# Komentar       # Baris diawali pagar dianggap komentar penjelasan
```

[PERHATIAN - Jebakan Umum]: .gitignore **hanya berlaku** untuk berkas yang belum pernah di-track. Jika berkas terlanjur masuk riwayat commit lama, menambahkannya ke .gitignore tidak akan otomatis menghapusnya. Lepaskan berkas dari pelacakan terlebih dahulu:
git rm --cached flask.log | git commit -m "chore: stop tracking flask.log"
Periksa status abaikan berkas dengan: git check-ignore -v <nama_file>

4. Rincian .gitignore pada Proyek Jamur

Pola Berkas	Alasan Teknis Diabaikan
__pycache__/ , *.pyc	Bytecode Python hasil kompilasi lokal otomatis, spesifik tiap mesin.
.DS_Store	File cache metadata internal Finder macOS, tidak relevan untuk repositori.
flask.log	Log lokal request server pengembangan Flask, berubah terus-menerus.
templates/index.old.html	Cadangan revisi HTML lama; tidak perlu mengotori cabang aktif.
templates/backup-old/	Direktori arsip eksplorasi desain terdahulu.
.venv/ , venv/	Direktori virtual environment Python (berukuran puluhan/ratusan MB).
.env	File variabel environment berisi kredensial rahasia (API key, token, password).

Catatan Model: Berkas mushroom_pipeline.joblib (5.4 KB) dan feature_categories.joblib **sengaja tetap di-commit** karena berukuran sangat kecil dan wajib ada agar aplikasi Vercel langsung berfungsi tanpa training ulang.

5. Aturan Baku Keamanan Kredensial

DILARANG KERAS meng-commit berkas .env, API token, private key, atau kata sandi database. Bila rahasia terlanjur ter-push ke GitHub publik, menghapusnya pada commit berikutnya **TIDAK MENYELESAIKAN MASALAH** karena riwayat lama (Git history) tetap dapat diakses publik.
Tindakan darurat: Segera *revoke* atau *rotate* token dari dashboard penyedia API, lalu bersihkan histori dengan git filter-repo.
Lakukan pengecekan mandiri sebelum push: git status | git diff --cached | grep -i -E "api|secret|token|password"

6. Alur Kerja Harian Git (Daily Workflow)

```
git status                  # Cek status berkas yang berubah
git add -A                  # Masukkan seluruh modifikasi ke staging area
git commit -m "pesan"       # Simpan snapshot lokal dengan pesan deskriptif
git push                    # Kirim commit lokal ke remote GitHub
git pull                    # Unduh dan gabungkan perubahan terbaru dari remote
git log --oneline           # Lihat riwayat commit secara ringkas
git diff                    # Periksa baris modifikasi yang belum di-stage
```

Standar Pesan Commit (Conventional Commits):
- feat: tambah tab game tebak jamur (fitur baru)
- fix: perbaiki marker anatomi tidak menyala (bugfix)
- chore: rapikan aturan abaikan berkas (pemeliharaan)

7. Fitur GitHub & Kolaborasi Tim

- **Visibilitas Repositori: Private:** hanya Anda dan kolaborator terundang yang dapat melihat. **Public:** dapat diakses dan di-clone publik.
- **Dokumentasi README.md:** Halaman muka repositori di GitHub; berisi panduan instalasi, dependensi, dan ringkasan metrik model.
- **Trik Berkas .gitkeep:** Git mengabaikan folder kosong; menaruh .gitkeep memastikan folder static/img/ tetap terbawa.
- **Alur Kolaborasi Tim (3 Orang):** Hindari commit langsung ke branch main bersamaan. Tiap anggota membuat branch kerja (misal: git checkout -b feature/model-knn), lalu mengajukan **Pull Request (PR)** untuk di-review sebelum di-merge.
- **Hosting: GitHub Pages vs Vercel:** GitHub Pages hanya melayani web statis murni. Untuk backend Python Flask / machine learning, deploy wajib menggunakan platform seperti **Vercel** atau **Render**.

8. Status & Langkah Eksekusi Repositori Proyek

Repositori lokal di direktori /Users/mac/mushroom-flask-app sudah siap sepenuhnya dengan dua commit resmi (dd01cdb dan 9d95518) di branch main.

```
# 1. Pindah ke direktori proyek:
cd /Users/mac/mushroom-flask-app

# 2. Hubungkan remote GitHub yang sudah dibuat:
git remote add origin https://github.com/goosonk/mushroom-ml-app.git

# 3. Unggah seluruh kode ke branch utama:
git push -u origin main

# Bila di masa mendatang ada perubahan kode / template:
git add -A
git commit -m "feat: perbarui visual antarmuka"
git push
```